

Programmiervorkurs 2026

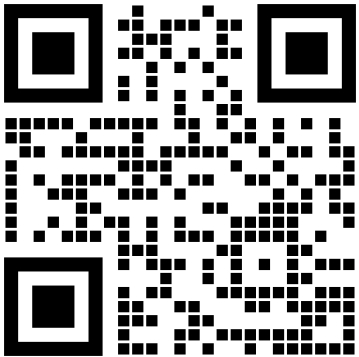
Rekursion

Vorkurs-Schnupsis

30. September 2026

TU Darmstadt

Fragen



fragen.d120.de

Wiederholung

Funktionen und Schleifen

Wiederholung

```python
def zaehlen(name, buchstabe):
  zaehler = 0
  for zeichen in name:
    if zeichen.lower() == buchstabe:
      zaehler += 1
  return zaehler

name = input('Name: ')
zahl = zaehlen(name, 'a')
print(f"Du hast {zahl} A's im Namen")
```

Funktionen und Schleifen

Wiederholung

- Kann man damit schon alles machen?

  $\Rightarrow$ Ja, aber es wird nicht schön...
- Tatsächlich können Funktionen noch einiges mehr
- Eine sehr wichtiges Eigenschaft wollen wir euch nicht vorenthalten:

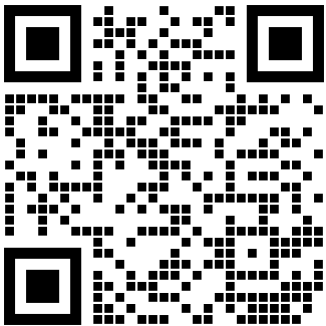
  $\Rightarrow$ Funktionen können sich selber aufrufen!

Evaluation

Evaluation

Bewerte unsere Vorlesung und Übung

Dafür habt ihr natürlich ein paar Minuten Zeit …

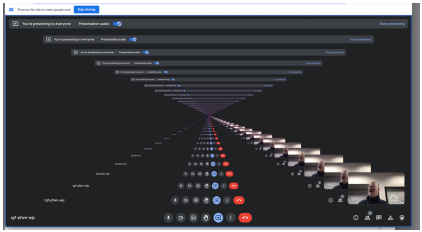


https://umfragen.tu-darmstadt.de/192179

Rekursion

Rekursion

Was ist das eigentlich?





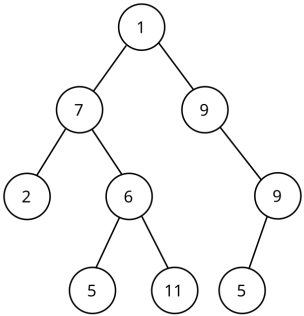
Rekursion im Alltag

Rekursion

Bruder, was das?



Rekursion in der Natur



Binärbäume in der Informatik

Rekursion

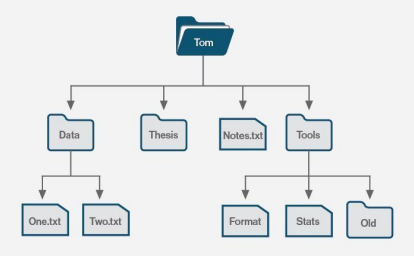
Was ist das eigentlich?

- Viele Dinge (insb. in der Mathematik oder Informatik) sind oft im Bezug auf sich selbst definiert
- Das nennt man eine *rekursive Definition*
- Beispiele dafür sind unter anderem:
  - Die Fibonacci-Folge
  - Die Fakultät
  - Der Programmcode, den wir schreiben
  - Dateisysteme

Rekursion

Was ist das eigentlich?

Ordner sind **rekursiv** definiert, d.h. diese können Unterordner haben



Rekursion in Dateisystemen

Definition

Wie definieren wir Rekursion?

Die wesentlichen Bestandteile:

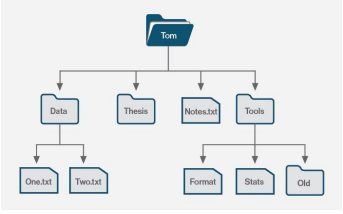
- Anfangs- bzw. Ausgangszustand,
  oft *Rekursionsanker* genannt
- Anweisungsabfolge, um von einem Zustand zum nächsten zu kommen,
  oft *Rekursionsschritt* genannt

Definition

Wie definieren wir Rekursion?

Beispiel Dateisystem:

- Szenario: Man möchte vom Wurzelknoten aus eine Datei suchen
- Dafür werden in jedem Ordner die Liste der Unterelemente durchlaufen und geprüft, ob die gesuchte Datei dabei ist
- Wurde die Datei gefunden oder es ist nicht möglich weiter abzusteigen, kehrt die Suchfunktion zur Ebene darüber zurück
- Überlegt euch in den davor gezeigten Beispielen, was Anker- und was Rekursionsschritt ist



Rekursion in Python

Wie definieren wir Rekursion in Python?

Die Fakultät ist rekursiv definiert als

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n > 0 \end{cases}$$

In Python:

```python
def factorial(n):
  if n == 0:   # Rekursionsanker
    return 1

  # Rekursiver Funktionsaufruf von factorial
  return n * factorial(n-1)
```

Rekursion in Python

```python
def is_even(n):
  if n == 0: # Rekursionsanker
    return True
  return is_odd(n-1) # Rekursiver Funktionsaufruf

def is_odd(n):
  if n == 0: # Rekursionsanker
    return False
  return is_even(n-1) # Rekursiver Funktionsaufruf
```

Merke: Rekursion kann auch hin und her springen

Rekursion in Python

Die Fibonacci Folge ist rekursiv definiert als

$$fib(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-1) + fib(n-2) & n > 1 \end{cases}$$

In Python:

```
1 def fibonacci(n):
2   if n <= 1:   # Rekursionsanker
3     return n
4   #      v -------------- v ------------ Rekursive Funktionsaufrufe
5   return fibonacci(n-1) + fibonacci(n-2)
```

Merke: Funktionen können sich auch **mehrmals** rekursiv aufrufen

Live-Coding

Rekursion in Python

Probleme

- Grundsätzlich lässt sich jede Schleife in Python auch mit Rekursion umsetzen
- Ausnahme: Endlosschleife bzw. Schleifen mit zu vielen Iterationen
- Python hat eine maximale Tiefe an Funktionsaufrufen von ungefähr 1000

Rekursion in Python

Probleme

Unendliche Rekursion

```
>>> def fun(num):
    fun(num+1)
>>> fun(0)
```

Was ist das Ergebnis?

Rekursion in Python

Probleme

Unendliche Rekursion

```
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    fun(0)
    ~~~^^^
  File "<python-input-0>", line 2, in fun
    fun(num+1)
    ~~~^^^^^^^
  [Previous line repeated 988 more times]
RecursionError: maximum recursion depth exceeded
```

Definition

Probleme

Live-Coding

Rekursion

Quiz

- Wie viele Rekursionsanker braucht man?
- Wie macht man eine Funktion in Python rekursiv?
- Ist im Allgemeinen Rekursion oder Iteration sinnvoller?

Java



Ein Ausblick I

Java ist die nächste Programmiersprache, die ihr im Studium lernen werdet. Aber wie unterscheidet sich Java von Python?

- Statische Typisierung
  - Typen von Variablen, Rückgabetypen und Funktionsparametern müssen im Quelltext stehen
  - Zur Laufzeit kann dadurch weniger schief gehen
    $\Rightarrow$ Eure Programme sind robuster

Ein Ausblick II

- Objektorientierung
  - Funktionen auf Objekten: **Methoden**
  - Diese kennt ihr bereits, z.B. `"Hallo".upper()`
- Einrückung ist in Java irrelevant
  - Stattdessen nutzt man geschweifte Klammern und Semikola, um Code zu trennen
- Java wird kompiliert
  - Man führt nicht direkt den Quelltext aus, sondern Bytecode
  - Vor der Ausführung muss erst der Compiler aufgerufen werden (`javac`)

Variablen

Python:

```
1 x = 5
2 s = 'Lorem'
```

Java:

```
1 int x = 5;
2 String s = "Lorem";
```

Wichtiger Unterschied:
In Java können Strings nur mit `""` definiert werden, nicht mit `''`

Verzweigungen

Python:

```python
if x < 0:
    print("Negativ")
elif x > 0:
    print("Positiv")
else:
    print("Null")
```

Java:

```java
if (x < 0) {
    System.out.println("Negativ");
} else if (x > 0) {
    System.out.println("Positiv");
} else {
    System.out.println("Null");
}
```

Schleifen

while

Python:

```python
while n > 0:
    n = n // 2 - 1
```

Java:

```java
while (n > 0) {
    n = n / 2 - 1;
}
```

Zusätzlich gibt es eine do-while Schleife, diese lernt ihr dann im ersten Semester.

Schleifen

for-each

Python:

```python
fruits = ["apple", "orage", "banana"]
for f in fruits:
    print(f + "tasted good")
```

Java:

```java
String[] fruits = {"apple", "orange", "banana"};
for (String f : fruits) {
    System.out.println(f + "tasted good");
}
```

Schleifen

for mit Zähler

Python:

```python
for i in range(1, 11):
    print(str(i) + " squared is " + str(i*i))
```

Java:

```java
for (int i = 1; i < 11; ++i) {
    System.out.println(i + " squared is " + i*i)
}
```

Funktionen

Python:

```python
def say_hello(name):
    print("Hello " + name + "!")
    print("How are you?")
```

Java:

```java
void sayHello(String name) {
    System.out.println("Hello " + name + "!");
    System.out.println("How are you?");
}
```

Software-Design

Guter Code

- Probleme kann man häufiger abstrakter betrachten, um sie allgemeiner (und einfacher) zu lösen oder auch weniger Code zu duplizieren

  $\Rightarrow$ Allgemeiner formulierte statt tausenden Bedingungen

  $\Rightarrow$ Letzteres sieht man leider immer wieder bei Anfängern
- Natürlich gilt: Nicht besonders ”klug” designen, ihr versteht es später selber nicht mehr

Guter Code

Beispiel: Verbose Version

```
1  def vier_waagerecht(x, y):
2    entry = grid[y][x]
3    for i in range(1, 4):
4      if grid[y][x+i] != entry:
5        return False
6    return True
7
8  def vier_senkrecht(x, y):
9    entry = grid[y][x]
10   for i in range(1, 4):
11     if grid[y+i][x] != entry:
12       return False
13   return True
```

Guter Code

Beispiel: Kompakte Version

```
1 def vier_in_folge(x, y, xo, yo):
2   entry = grid[y][x]
3   for i in range(1, 4):
4     if grid[y+i*yo][x+i*xo] != entry:
5       return False
6   return True
```

Mit zwei zusätzlichen Parametern braucht man nur noch eine Funktion, die Logik der beiden sollte sich abgesehen von der Richtung auch identisch verhalten. Zusätzlich ist so auch eine Prüfung der Diagonalen möglich.

(Mangelnde) Effizienz

- In diesem Vorkurs (und dem ersten Semester) ist das natürlich noch nicht so wichtig
- Man sollte sich dennoch beim Grund-Design bereits Gedanken dazu machen
- Windows, Discord, … ist alles nicht sonderlich effizient, Rechner könnten heutzutage eigentlich deutlich schneller sein
- Beispiele:
  - Funktionsaufrufe, welche in einer Schleife immer das gleiche Ergebnis liefern, sollte man herausziehen
  - Für zeilenweises Bearbeiten einer Datei: Nicht für jede Zeile die Datei öffnen, schließen und die n-te Zeile ändern/lesen

Sprachensalat



Funktional

Haskell Brooks Curry

- Verkettung von Funktionen ist die Grundlage für komplexe Programme
- Es gibt keine Variablen, sondern nur Konstanten
- Funktionen sind wie Werte und können als Rückgabewerte und Parameter verwendet werden
- Kombinatoren können als weitere Verkettung von Funktionen verwendet werden
- Häufig können Funktionen auch nur partiell angewendet werden (Currying)

Stack basierte Sprachen

Forth was discovered not invented - Charles H. Moore

- Grundprinzip: Es gibt einen oder mehrere Stacks für Daten
- Operatoren arbeiten nur auf Stacks (Parameter, Rückgabewerte, etc.)
- Funktionen werden als Abfolge von Operatoren auf den Stack abgelegt
- Sehr hardwarenah, da so gut wie jede CPU eine Stack-Maschine ist/beinhaltet
- Funktionen sind Adressen (und damit Zahlen), die auf den Stack gelegt werden können
- Es gibt keine Arrays, Schleifen, Listen oder Sonstiges, es gibt nur den STACK

STACK
STACK
STACK
STACK

A Programming Language (APL)

Arrays are also primitive values

- Alles basiert auf N-Dimensionalen Arrays, auch Zahlen sind nur 0D-Arrays
- Operatoren arbeiten auf Arrays und kombinieren diese zu anderen Arrays
- Wie in funktionalen Sprachen können Funktionen kombiniert werden, da quasi jede Funktion dieselbe Signatur hat und nur eine andere Dimension
- Funktionen können fast magisch über N-Dimensionen angewendet werden, auch wenn sie nicht dafür gedacht waren
- Besteht im Wesentlichen aus einzelnen Symbolen oder Keywords

Game of Life:

```
life ← {⊃1 ⍵ ∨.∧ 3 4 = +/ +⌿ ¯1 0 1 ∘.⊖ ¯1 0 1 ⌽¨ ⊂⍵}
```

AsciiDots

Enter 2D Programming

- Nicht alles an Code muss zeilenbasiert sein
- Es können Zusammenhänge zwischen Werten effizient dargestellt werden
- Beschränkt sich auf primitive Operationen und überlässt dem Programmierer das Kopieren/Duplizieren
- Kann kompiliert werden zu einem Graphen, der die Operationen als eine sequentielle Logikkette darstellt
- Schleifen werden als Kreise dargestellt
- Es gibt keine expliziten Variablen, nur Werte

```
.-#1\
.  /->\
>[+]  |
\-*#$/
```

Fibonacci in AsciiDots

Piet

Farben können auch Dinge tun

- Besteht aus 20 Farben (18 Farben sowie Schwarz und Weiß)
- Bringt einen zum Nachdenken, da es schon für einfache Probleme wie ein Puzzle ist
- Damit sind auch manche Bilder Turing-complete, wenn man sie richtig interpretiert
- Man hat ein schönes Bild am Ende



Hello World in Piet

Key Take-Aways

Es gibt verschiedene Paradigmen, welche Probleme anhand von ihren Eigenschaften verschieden gut lösen können.
Weiterhin schließen sie sich nicht direkt aus, sondern leben friedlich zusammen in einem N-Dimensionalen Raum auf orthogonalen Achsen.



Ende

Vielen Dank

Viel Spaß in der Orientierungswoche und im weiteren Studium!

Nachher ist noch eine Übung und morgen Zeit für die Programmierchallenge (ansonsten genießt das lange Wochenende).

Denkt an die Challenge, wir freuen uns über **jede** Abgabe.